

System Design Document (SDD)

MotorsRent

Riferimento	
Versione	1.1
Data	10/11/2025
Destinatario	Azienda MotorsRent S.r.l.
Presentato da	Ingenito Davide Sorrentino Luca Teodonno Francesco
Approvato da	

Revision History

Data	Versione	Descrizione	Autori
10/11/2025	0.1	Design Goals	Team
12/11/2025	0.2	Decomposizione in sottosistemi	Team
12/11/2025	0.3	Deployment Diagram	Team
19/11/2025	0.4	Persistent Data Management	Team
20/11/2025	0.5	Controllo accessi e controllo globale del software	Team
21/11/2025	1.0	Boundary condition e servizi dei sottosistemi	Team
23/11/2025	1.1	Revisione del documento	Team

Revision History.....	2
1. Introduzione.....	4
1.1 Scopo del sistema	4
1.2 Design goals.....	4
1.3 Definizioni, acronimi, e abbreviazioni.....	5
2. Architettura del sistema corrente	6
3. Architettura del sistema proposto.....	6
3.1 Overview	6
3.2 Decomposizioni sottosistemi in Layer.....	7
3.3 Deployment Diagram.....	10
3.4 Mapping hardware / software	10
3.5 Persistent Data Management	12
3.5.1 Modellazione delle entità persistenti.....	12
3.5.2 Modellazione delle relazioni.....	13
3.5.3 Considerazioni sulle funzionalità applicative	13
3.5.4 Integrità dei dati	13
3.6 Controllo accessi	16
3.7 Controllo Globale del Software	17
3.8 Boundary Conditions	17
3.9 Servizi dei sottosistemi.....	20

1. Introduzione

1.1 Scopo del sistema

Il sistema MotorsRent nasce con l'obiettivo di digitalizzare e rendere più efficiente la gestione di una concessionaria automobilistica, offrendo una piattaforma che metta in contatto l'azienda con utenti interessati all'acquisto o al leasing di un veicolo.

La piattaforma consente la registrazione di clienti, venditori e amministratori, ciascuno dotato di strumenti specifici. Gli utenti possono consultare il catalogo dei veicoli e inoltrare richieste, mentre il personale della concessionaria può gestire l'offerta e rispondere alle domande in modo organizzato e centralizzato.

Il sistema integra un catalogo digitale delle auto, che consente agli utenti di consultare l'inventario tramite filtri avanzati, visualizzare le schede dettagliate dei veicoli e inviare richieste di preventivo o leasing, insieme a un'area gestionale interna destinata a venditori e amministratori, attraverso la quale è possibile aggiornare il catalogo, monitorare le richieste ricevute e predisporre offerte personalizzate per ciascun cliente. Per facilitare la scelta del veicolo e migliorare l'esperienza dell'utente, MotorsRent include anche un calcolatore di leasing, che offre una stima immediata della rata mensile sulla base dei parametri forniti.

Nel suo complesso, il sistema mira a semplificare il processo di contatto tra cliente e concessionaria, migliorare la fruibilità del catalogo e rendere più fluida la gestione interna delle attività commerciali.

1.2 Design Goals

Rank	ID	Descrizione	Categoria	RNF di origine
4	DG_P_1	Progettare un'architettura ottimizzata per ridurre i tempi di caricamento del catalogo veicoli, ad esempio tramite caching e caricamento asincrono delle risorse.	Prestazioni	RNF_P_1
5	DG_P_2	Implementare una gestione efficiente delle connessioni e delle risorse server per supportare almeno 50 utenti simultanei senza degrado delle prestazioni.	Prestazioni	RNF_P_2
6	DG_A_1	Progettare un'interfaccia utente con layout chiaro e navigazione intuitiva, minimizzando i passaggi necessari per compiere le operazioni principali.	Affidabilità	RNF_A_1
7	DG_A_2	Adottare un design responsive che garantisca il corretto adattamento dell'interfaccia a diversi dispositivi e risoluzioni (desktop, tablet, smartphone).	Affidabilità	RNF_A_2
8	DG_A_3	Utilizzare combinazioni di colori, dimensioni dei caratteri e contrasto adeguati ad assicurare una leggibilità ottimale dei testi.	Affidabilità	RNF_A_3
1	DG_U_1	Implementare meccanismi di monitoraggio e ridondanza per garantire una disponibilità minima del 95% durante l'orario di utilizzo.	Usabilità	RNF_U_1
2	DG_U_2	Integrare un sistema di gestione degli errori che mostri messaggi chiari all'utente e impedisca il crash del sistema.	Usabilità	RNF_U_2
3	DG_U_3	Progettare un sistema di salvataggio persistente dei dati utente (es. database transazionale o salvataggio temporaneo) per evitare perdite in caso di crash o timeout.	Usabilità	RNF_U_3

1.3 Definizioni, acronimi e abbreviazioni

La presente sezione raccoglie le principali definizioni, gli acronimi e le abbreviazioni utilizzati nel documento, con l'obiettivo di garantire uniformità terminologica e chiarezza nella consultazione dello SDD. Le definizioni qui riportate si riferiscono a concetti, ruoli, componenti architetturali e tecnologie direttamente impiegate nel sistema MotorsRent.

Definizioni

- **MotorsRent** – Sistema informativo web progettato per digitalizzare e ottimizzare la gestione delle attività di una concessionaria automobilistica, includendo catalogo veicoli, richieste di preventivo, simulazioni e richieste di leasing, oltre alla gestione interna di venditori e amministratori.
- **Ospite** – Utente non autenticato che può esclusivamente consultare il catalogo dei veicoli e procedere alla registrazione.
- **Cliente** – Utente registrato che può consultare il catalogo, inviare richieste di preventivo o leasing, monitorarne lo stato e gestire i dati del proprio profilo.
- **Venditore** – Utente interno dell'azienda, incaricato della gestione delle richieste dei clienti, della preparazione dei preventivi e della valutazione o modifica delle richieste di leasing.
- **Amministratore** – Utente con privilegi massimi, responsabile della gestione del catalogo, dell'amministrazione dei venditori e della supervisione generale del sistema.
- **Catalogo Veicoli** – Raccolta digitale centralizzata dei veicoli disponibili, arricchita da filtri di ricerca, schede tecniche dettagliate e funzionalità di consultazione.
- **Preventivo** – Proposta economica formalizzata dal venditore in risposta alla richiesta di un cliente e associata a un veicolo presente nel catalogo.
- **Richiesta di Leasing** – Procedura avviata dal cliente per ottenere una simulazione o un'offerta di leasing sulla base di parametri quali durata, anticipo e chilometraggio annuo.
- **MVC (Model–View–Controller)** – Pattern architetturale adottato dal sistema, che separa responsabilità di presentazione (View), logica applicativa (Controller) e gestione dei dati e delle regole di business (Model).
- **Repository Architecture** – Stile architetturale in cui tutti i dati persistenti sono gestiti tramite un deposito centralizzato, sfruttato da più componenti applicativi.
- **Three-Tier Architecture** – Architettura del sistema suddivisa in tre livelli distinti: Client (interfaccia), Web Server (logica applicativa), Database Server (gestione dei dati).
- **DBMS** – Software per la gestione e la manipolazione dei dati persistenti; in MotorsRent è utilizzato per conservare veicoli, utenti, preventivi e richieste di leasing.
- **Web Container** – Ambiente di esecuzione per componenti web server-side, responsabile della gestione dei controller e della generazione dinamica degli elementi dell'interfaccia.

Acronimi e abbreviazioni

- **SDD** – *System Design Document*, documento destinato alla descrizione tecnica dell'architettura e del design di sistema.
- **MVC** – *Model–View–Controller*, pattern architetturale che separa presentazione, logica applicativa e gestione dei dati.
- **DBMS** – *Database Management System*, sistema per la gestione del database.
- **DB** – *Database*, archivio strutturato dei dati persistenti.

- **UI** – *User Interface*, interfaccia utente.
- **HTTP/HTTPS** – *HyperText Transfer Protocol / Secure*, protocolli utilizzati per la comunicazione tra client e server.
- **JDBC** – *Java Database Connectivity*, driver e API per la comunicazione tra applicazione e database relazionale.
- **ER** – *Entity-Relationship*, schema utilizzato per la modellazione dei dati persistenti.
- **RNF** – *Requisito Non Funzionale*, tipo di requisito che definisce qualità, vincoli e metriche del sistema.
- **DG** – *Design Goal*, obiettivo progettuale derivato da uno o più requisiti non funzionali.
- **UC** – *Use Case*, caso d'uso che descrive un'interazione tipica tra attore e sistema.

2. Architettura del Sistema Corrente

Attualmente non esiste una piattaforma che soddisfi gli obiettivi funzionali e operativi previsti dal progetto *MotorsRent*. Il dominio applicativo rientra quindi nella **Green-field Engineering**, poiché il sistema introduce per la prima volta funzionalità integrate di consultazione del catalogo, richiesta preventivi, richiesta leasing e gestione centralizzata delle attività da parte del personale della concessionaria.

Un sistema parzialmente comparabile è rappresentato dalle comuni piattaforme delle concessionarie (ad esempio Arval), che presentano alcune funzionalità sovrapponibili, quali:

- visualizzazione del catalogo veicoli;
- consultazione delle caratteristiche tecniche;
- reperimento dei contatti della concessionaria.

Tuttavia, tali soluzioni **non forniscono** un flusso digitale completo per la gestione delle richieste di preventivo o leasing. La formalizzazione del contratto, infatti, avviene generalmente al di fuori della piattaforma e richiede interazione fisica tra cliente e consulente.

MotorsRent si differenzia introducendo una **gestione online strutturata** che consente al cliente di:

- inviare richieste di preventivo direttamente dalla scheda del veicolo;
- richiedere simulazioni di leasing;
- ricevere una risposta formale da un venditore tramite area riservata;
- consultare e monitorare l'avanzamento delle proprie richieste.

3. Architettura del Sistema Proposto

3.1 Overview

Il sistema proposto per *MotorsRent* rientra pienamente nella **Green-field Engineering**, poiché introduce per la prima volta un ambiente web integrato dedicato alla gestione digitale del catalogo e delle operazioni commerciali correlate (preventivi e leasing).

La piattaforma si rivolge a:

- *Ospiti*, che possono consultare il catalogo;
- *Clienti registrati*, che possono richiedere preventivi e contratti di leasing;
- *Venditori*, che gestiscono le richieste ricevute;
- *Amministratori*, che gestiscono utenti interni e catalogo veicoli.

Utente Ospite

L'utente non autenticato può:

- consultare il catalogo;
- visualizzare le schede dei veicoli;
- registrarsi alla piattaforma.

Non può, invece, inviare richieste di preventivo o leasing.

Cliente Registrato

Il Cliente può:

- consultare il catalogo;
- inviare richieste di preventivo;
- inviare richieste di leasing;
- visualizzare lo stato delle proprie richieste;
- accedere alla propria area personale per modificare i dati.

Venditore

Il Venditore (creato dall'Amministratore) può:

- accedere all'area riservata;
- visualizzare tutte le richieste di preventivo e leasing assegnate o non ancora gestite;
- elaborare preventivi;
- confermare, modificare o respingere richieste di leasing;
- visualizzare l'elenco clienti;
- consultare il catalogo veicoli.

Amministratore

L'Amministratore dispone delle massime autorizzazioni e può:

- accedere all'area riservata;
- aggiungere, modificare o rimuovere veicoli dal catalogo;
- aggiungere o rimuovere Venditori;
- controllare le statistiche e il corretto funzionamento della piattaforma.

3.2 Decomposizione in sottosistemi in Layer

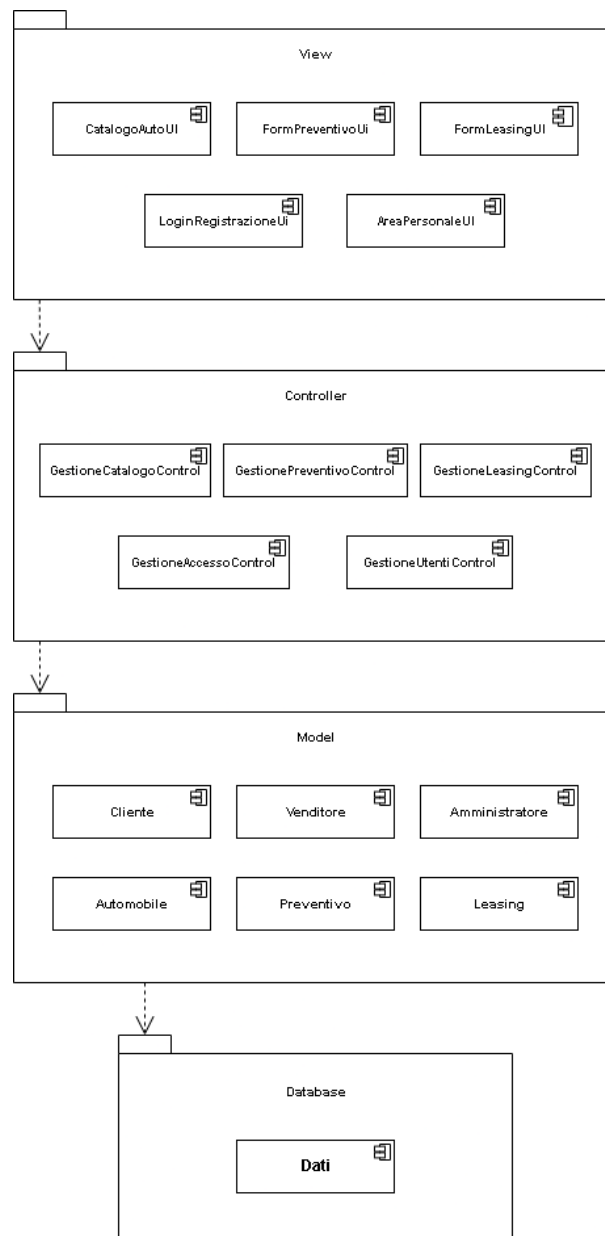
Per la decomposizione del sistema in sottosistemi è stato adottato il pattern architetturale MVC (Model–View–Controller).

Questa scelta consente di ottenere una struttura modulare, facilmente estendibile e con un'elevata separazione delle responsabilità tra le varie componenti del sistema.

Il pattern MVC è stato selezionato in quanto si adatta perfettamente a un'applicazione web come *MotorsRent*, che richiede un'interazione continua tra interfaccia utente, logica applicativa e gestione dei dati.

In particolare:

- Il layer **View** gestisce la parte di presentazione del sistema, ovvero tutte le interfacce con cui l'utente interagisce (consultazione del catalogo, compilazione dei moduli di richiesta, area personale, ecc.).
- Il layer **Controller** funge da intermediario tra la vista e il modello, ricevendo gli input dell'utente e coordinando le operazioni da eseguire. Questo garantisce una logica di controllo centralizzata e facilita la manutenzione del codice.
- Il layer **Model** racchiude la logica di business e la gestione dei dati. Qui risiedono le entità principali del dominio (ad esempio *Utente*, *Automobile*, *Preventivo*, *Leasing*), oltre ai meccanismi di persistenza verso il database.



Layer & Sottosistemi MVC per MotorsRent

Layer “**View**” (Presentazione / Interfaccia Utente)

- Sottosistemi:
 - **CatalogoAutoUI** — visualizzazione catalogo auto, filtri, scheda dettagli
 - **FormPreventivoUI** — modulo richiesta preventivo, visualizzazione offerta
 - **FormLeasingUI** — modulo simulazione/leasing
 - **LoginRegistrazioneUI** — interfaccia accesso e creazione account
 - **AreaPersonaleUI** — dashboard cliente, storico richieste, stato preventivo/leasing

Layer “**Controller**” (Logica di Controllo)

- Sottosistemi:
 - **GestioneCatalogoController** — riceve input da UI catalogo, invoca filtraggio, invia dati
 - **GestionePreventivoController** — gestisce richiesta preventivo, notifica venditore, comunica al cliente
 - **GestioneLeasingController** — gestisce richiesta leasing, calcolo rata, proposta al cliente
 - **GestioneUtentiController** — login, registrazione, profilo, permessi
 - **GestioneAmministrazioneController** — operazioni di amministratore: gestione catalogo, utenti, ruoli

Layer “**Model**” (Persistenza)

- Sottosistemi:
 - **Cliente**
 - **Venditore**
 - **Amministratore**
 - **Automobile**
 - **Preventivo**
 - **Leasing**
 - **Persistenza**

3.3 Deployment Diagram

L'architettura del sistema *MotorsRent* è organizzata secondo un modello a tre livelli (three-tier architecture), basato sul pattern MVC (Model-View-Controller), che assicura modularità, scalabilità e una chiara separazione dei compiti.

Il **Client Utente** rappresenta la componente front-end del sistema, accessibile tramite un **browser web**. Attraverso il browser, l'utente può navigare nel portale, consultare il catalogo dei veicoli e interagire con le funzionalità offerte, come la richiesta di preventivo o di leasing. La comunicazione con il server avviene mediante il protocollo **HTTP/HTTPS**, garantendo sicurezza e affidabilità nello scambio dei dati.

Il **Web Server** ospita i moduli **Controller** e **View**:

- Il *Controller* gestisce la logica applicativa e coordina le interazioni tra utente e sistema.
- La *View* è responsabile della presentazione delle informazioni e dell'interfaccia utente.

Il **Database Server** contiene il componente **Database**, che memorizza tutte le informazioni relative a utenti, veicoli, preventivi e leasing. La comunicazione tra il Web Server e il Database Server avviene tramite connessione **JDBC/SQL**, che consente l'interrogazione e l'aggiornamento efficiente dei dati.

Questa struttura distribuita garantisce una gestione efficiente delle risorse, una migliore manutenibilità del sistema e la possibilità di scalare i singoli componenti in base alle esigenze operative.

3.4 Mapping hardware/software

L'architettura di *MotorsRent*, si basa su un modello distribuito a tre livelli. Questo permette di separare chiaramente le responsabilità tra interfaccia utente, logica applicativa e gestione dei dati.

1. Il Client Utente

Il primo elemento dell'architettura è il **client dell'utente**, che può essere un normale PC, un tablet o uno smartphone dotato di browser.

Su questo dispositivo non risiede alcuna logica applicativa avanzata: il suo compito è visualizzare l'interfaccia e inviare richieste al server.

Tutti i componenti del **layer View**, come:

- la pagina del catalogo delle auto,
- i moduli per la richiesta di preventivo o leasing,
- l'interfaccia di login e registrazione,
- l'area personale del cliente,

vengono renderizzati qui e rappresentano l'interfaccia con cui l'utente interagisce.

Il browser è quindi il punto in cui l'utente percepisce il sistema, anche se la logica che regola tali interfacce è ospitata altrove.

2. Il Web Server

Il secondo nodo è il **Web Server**, che rappresenta il centro nevralgico dell'applicazione.

Qui risiedono sia le componenti del **Controller**, responsabili della logica di controllo, sia quelle della **View**, nella parte in cui viene generata o gestita la presentazione dei contenuti (prima che vengano inviati al client).

All'interno del Web Server vengono eseguite le operazioni principali del sistema:

- la gestione del catalogo,
- la generazione e gestione dei preventivi,
- la simulazione del leasing,
- la gestione degli utenti,
- le funzionalità di amministrazione.

In questo nodo si coordinano tutte le interazioni tra la UI del client e i dati memorizzati nel database.

Il Web Server riceve le richieste del browser, le interpreta tramite i controller, recupera o aggiorna i dati necessari e infine costruisce la risposta da inviare al client.

La comunicazione con gli altri componenti avviene tramite protocollo sicuro **HTTPS** verso il client e tramite connessione **JDBC/SQL** verso il Database Server.

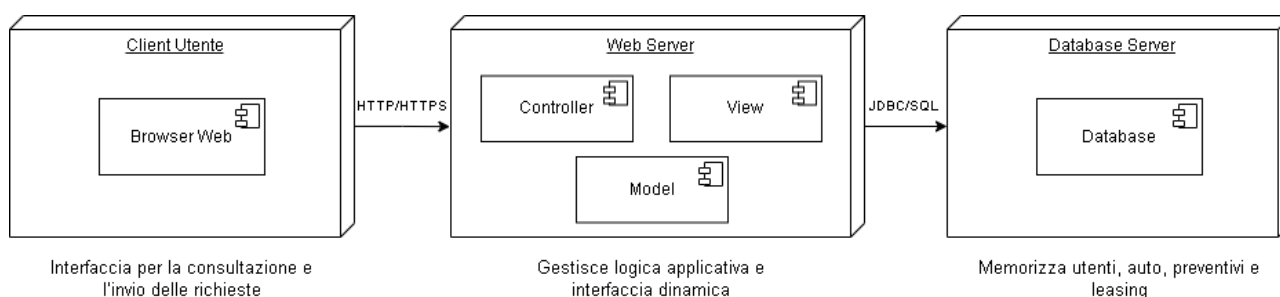
3. Il Database Server

Il terzo livello dell'architettura è il **Database Server**, dedicato esclusivamente alla gestione dei dati persistenti. Qui risiedono tutte le entità del **Model**, come:

- UTENTE,
- AUTOMOBILE,
- RICHIESTA_PREVENTIVO,
- RICHIESTA_LEASING.

Questo nodo è responsabile non solo della memorizzazione dei dati, ma anche dell'integrità referenziale tra le entità. Il Web Server interroga il Database Server ogni volta che è necessario ottenere o salvare informazioni: per esempio, quando un utente invia una richiesta di leasing o quando un venditore consulta le richieste ricevute.

Il Database Server non interagisce direttamente con gli utenti: si limita a operare come deposito strutturato e affidabile dei dati, garantendo consistenza e integrità tramite chiavi primarie, chiavi esterne e vincoli.



3.5 Persistent Data Management

Il sistema MotorsRent richiede una gestione strutturata e persistente dei dati allo scopo di supportare funzionalità di catalogo, interazione utente e processi di richiesta preventivi e leasing.

La persistenza dei dati è garantita attraverso un modello relazionale organizzato in quattro entità principali: **Utente**, **Auto**, **Richiesta Preventivo** e **Richiesta Leasing**.

Il database è progettato per garantire integrità referenziale, coerenza dei dati, performance e semplicità di consultazione da parte delle componenti applicative.

3.5.1 Modellazione delle entità persistenti

UTENTE

L'entità UTENTE contiene tutti i soggetti che interagiscono con il sistema (clienti, venditori, amministratori). Il ruolo è gestito come attributo e determina i permessi applicativi, senza introdurre differenze strutturali nel modello.

Questa scelta consente:

- gestione semplificata del login
- un'unica tabella anagrafica
- maggiore mantenibilità

AUTOMOBILE

Contiene i dati relativi alle vetture disponibili nel catalogo online.

Questa entità è fondamentale per le funzionalità di consultazione, filtraggio e selezione dell'auto da associare alle richieste.

RICHIESTA PREVENTIVO

Raccoglie tutte le richieste di preventivo inviate dai clienti.

Ogni richiesta è associata a:

- un UTENTE (cliente)
- un'AUTOMOBILE del catalogo

I venditori possono visualizzare e gestire tali richieste tramite interfacce applicative

RICHIESTA LEASING

Memorizza le richieste di leasing effettuate dai clienti.

Anche in questo caso, ogni richiesta mantiene un collegamento stabile con un cliente e con un'auto.

I parametri tipici (durata, anticipo, chilometri annui) vengono salvati per permettere simulazioni, verifiche e conferme dell'offerta.

3.5.2 Modellazione delle relazioni

Il sistema utilizza quattro relazioni fondamentali, tutte di tipo **uno-a-molti (1:N)**, tra UTENTE/AUTOMOBILE e le richieste.

Le cardinalità sono le seguenti:

Relazione	Cardinalità	Significato
UTENTE → RICHIESTA_PREVENTIVO	(0,N) — (1,1)	Un cliente può inviare più richieste di preventivo
UTENTE → RICHIESTA_LEASING	(0,N) — (1,1)	Un cliente può inviare più richieste di leasing
AUTOMOBILE → RICHIESTA_PREVENTIVO	(0,N) — (1,1)	Una stessa auto può essere oggetto di più preventivi
AUTOMOBILE → RICHIESTA_LEASING	(0,N) — (1,1)	Una stessa auto può essere richiesta in più leasing

3.5.3 Considerazioni sulle funzionalità applicative

Alcune funzionalità del sistema, come:

- visualizzazione statistiche
- gestione da parte dei venditori
- dashboard amministratore

non introducono nuove entità persistenti.

Tali funzionalità operano tramite query e aggregazioni sui dati già memorizzati.

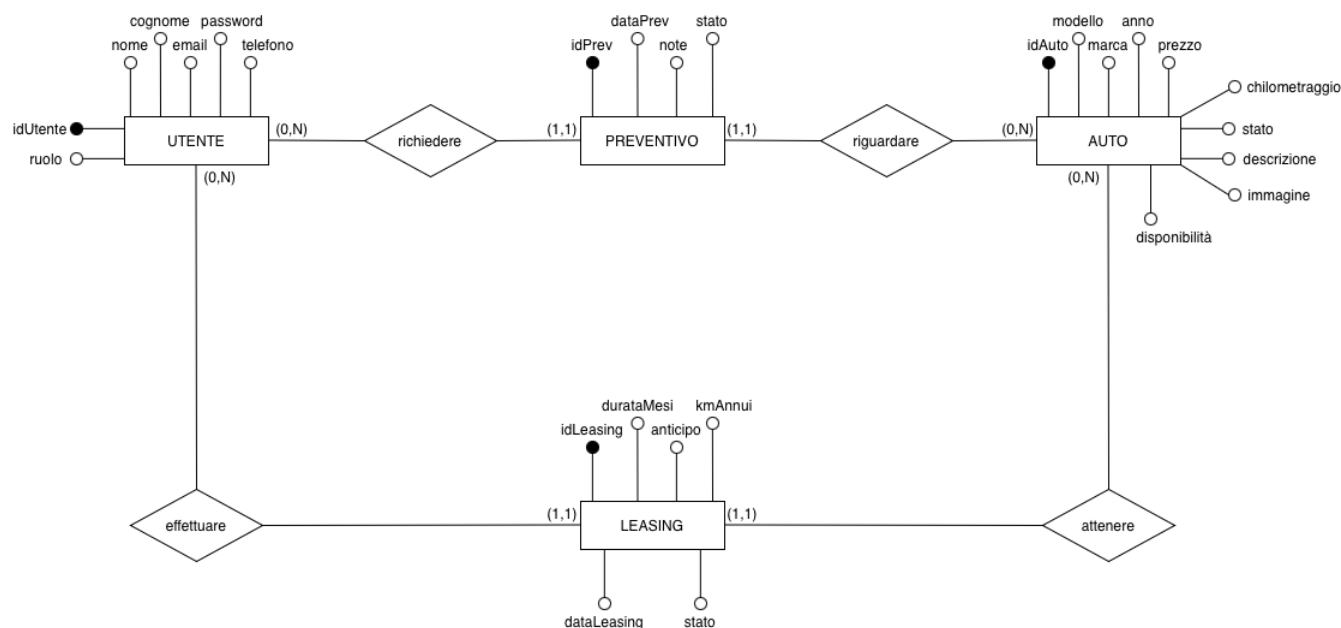
Questo consente un modello dati snello, ottimizzato e facilmente estendibile.

3.5.4 Integrità dei dati

Il sistema adotta:

- chiavi primarie su ogni entità
- chiavi esterne per garantire l'integrità referenziale
- regole di obbligatorietà per garantire la consistenza delle richieste
- vincoli sui ruoli per gestire i diritti applicativi

SCHEMA ER



Dizionario dei dati

Questa sezione descrive le entità presenti nel modello concettuale del sistema, specificandone caratteristiche principali, attributi e vincoli associati.

Nome entità	Utente		
Descrizione	Contiene le informazioni essenziali sugli utenti registrati		
Nome campo	Tipo	Vincolo di chiave	Altri vincoli
idUtente	Int	Primary key	Not nul
nome	Varchar(50)		Not null
cognome	Varchar(50)		Not null
email	Varchar(300)		Not null
password	Varchar(50)		Not null
telefono	Int		Not null
ruolo	Enum('Ospite', 'Cliente', 'Venditore', 'Amministratore')		Default 'Cliente'

Nome entità	Automobile		
Descrizione	Descrive le principali caratteristiche delle auto disponibili		
Nome campo	Tipo	Vincoli di chiave	Altri vincoli
idAuto	int	Primary key	Not null
modello	Varchar(50)		Not null
marca	Varchar(50)		Not null
anno	int		Not null
prezzo	Decimal(12,2)		Not null
chilometraggio	int		Not null
stato	Varchar(50)		Not null
descrizione	text		Not null
immagine	Varchar(255)		Not null
disponibilità	boolean		Not null

Nome entità	Leasing		
Descrizione	Memorizza i dati dei contratti di leasing associati alle auto		
Nome campo	Tipo	Vincoli di chiave	Altri vincoli
idLeasing	Int	Primary key	Not null
IdUtente	Int	Foreign key	Not null
idAuto	Int	Foreign key	Not null
durataMesi	Int		Not null
anticipo	Decimal(12,2)		Not null
kmAnnui	Int		Not null
rataMensile	Decimal(12,2)		Not null
dataRichiesta	DATETIME		Default Current_Timestamp
stato	Varchar(50)		Default 'In attesa'

Nome entità	Preventivo		
Descrizione	Raccoglie i dati relativi alle richieste di preventivo per un'auto		
Nome campo	Tipo	Vincoli di chiave	Altri vincoli
idPreventivo	Int	Primary key	Not null
idUtente	Int	Foreign key	Not null
idAuto	Int	Foreign key	Not null
dataRichiesta	DATETIME		Default Current_Timestamp
note	text		Not null
stato	Varchar(50)		Not null

3.6 Controllo accessi

La seguente matrice degli accessi definisce in modo strutturato i permessi associati ai diversi attori del sistema *MotorsRent*. L'obiettivo è identificare con precisione quali funzionalità ogni ruolo può visualizzare o modificare, garantendo così un adeguato livello di sicurezza, coerenza operativa e controllo degli accessi all'interno della piattaforma.

- La prima colonna riguarda il ruolo con privilegi minimi (*Ospite*), che può principalmente consultare informazioni pubbliche o avviare processi di base. Le **ultime** raggruppano invece i ruoli con maggiori responsabilità (*Cliente*, *Venditore*, *Amministratore*), che dispongono di permessi più ampi relativi alla gestione di profilo, veicoli, preventivi e leasing.

Attori	Ospite	Cliente	Venditore	Amministratore
Oggetti				
RegistrazioneUtente		CreaAccount		
GestioneAutenticazione		Login Logout	Login Logout	Login Logout
GestioneProfilo		VisualizzaProfilo ModificaProfilo	VisualizzaProfilo ModificaProfilo	VisualizzaProfilo ModificaProfilo EliminaProfilo
GestioneCatalogo	VisualizzaCatalogo	VisualizzaCatalogo	VisualizzaCatalogo	AggiungiVeicolo ModificaVeicolo EliminaVeicolo VisualizzaCatalogo
GestionePreventivo		VisualizzaPreventivo RichiediPreventivo	CreaPreventivo VisualizzaPreventivo ModificaPreventivo	
GestioneLeasing		RichiediLeasing VisualizzaLeasing	CreaLeasing VisualizzaLeasing ModificaLeasing	

3.7 Controllo Globale del Software

Il controllo globale del software in *MotorsRent* è strutturato attorno a un meccanismo **event-driven**, tipico delle web-application, in cui ogni azione dell'utente viene propagata attraverso un flusso ordinato di eventi e gestori. Ogni azione parte dall'interfaccia grafica: quando l'utente seleziona un comando, l'interfaccia genera un evento che viene intercettato da un **metodo di gestione della richiesta**, ossia il componente che si occupa di elaborare l'azione dell'utente e inoltrarla al sistema.

Il Controller riceve l'evento, ne verifica la validità, i parametri, i permessi associati al ruolo dell'utente e la sua coerenza con le regole applicative. Una volta validata, la richiesta viene indirizzata alla logica di business o ai servizi che implementano le funzionalità richieste. Quando è necessario accedere ai dati persistenti, il Controller comunica con il Model in maniera strutturata e sincrona, assicurando l'integrità degli scambi e impedendo accessi imprevisti o manipolazione impropria delle informazioni.

La gestione degli errori è centralizzata: eventuali eccezioni sollevate durante l'elaborazione vengono intercettate da un modulo dedicato, registrate nei log di sistema e convertite in messaggi chiari e non tecnici, così da mantenere la sicurezza interna e migliorare l'esperienza utente. Le operazioni critiche (ad esempio la creazione di una richiesta di preventivo o leasing) vengono inoltre eseguite in transazioni atomiche, così da evitare inconsistenze in caso di interruzioni o malfunzionamenti.

Per garantire stabilità anche in condizioni di carico elevato, il sistema supporta anche l'accesso concorrente da parte di più utenti grazie al connection pooling e ai livelli di isolamento delle transazioni. Infine, il sistema genera log dettagliati sugli accessi e sulle operazioni più importanti, permettendo agli amministratori di monitorare lo stato dell'applicazione e garantire stabilità e continuità del servizio.

3.8 Boundary Conditions

1. AVVIO DEL SISTEMA

Identificativo <i>UC_BC_1</i>	<i>System Start-up</i>	<i>Data</i>	<i>27/11/2025</i>
		<i>Vers.</i>	<i>0.00.001</i>
		<i>Autore</i>	<i>Team</i>
Descrizione	<i>Questo caso d'uso definisce la procedura attraverso la quale il sistema viene avviato.</i>		
Attore Principale	Amministratore		
Attori secondari	N.a.		
Entry Condition	L'amministratore ha effettuato l'accesso alla macchina server.		
Exit condition On success	Il sistema viene avviato correttamente.		
Exit condition On failure	Il sistema non viene avviato.		
FLUSSO DI EVENTI PRINCIPALE/MAIN SCENARIO			
1	Amministratore:	Avvia il DBMS per accedere ai dati contenuti nel database.	
2	Sistema:	Esegue la procedura di inizializzazione del DBMS.	

3	Sistema:	Conferma all'amministratore l'avvio del DBMS tramite un messaggio informativo.
4	Amministratore:	Procede all'attivazione del web container tramite la funzione predisposta.
5	Sistema:	Carica il web container e lo rende attivo.
6	Sistema:	Mostra un avviso che certifica l'avvio corretto del web container.
I Scenario/Flusso di eventi di ERRORE: Fallimento avvio DBMS		
2.a1	Sistema	Passa automaticamente all'esecuzione del caso d'uso UC_BC_3.
II Scenario/Flusso di eventi di ERRORE: Fallimento avvio web container		
5.a1	Sistema	Richiama il caso d'uso UC_BC_3 per gestire l'errore.

2. SPEGNIMENTO DEL SISTEMA

Identificativo <i>UC_BC_2</i>	<i>System shut-down</i>		<i>Data</i>	<i>27/11/2025</i>
			<i>Vers.</i>	<i>0.00.001</i>
			<i>Autore</i>	<i>Team</i>
Descrizione	<i>Lo UC descrive la procedura con cui l'amministratore spegne MotorsRent in modo sicuro.</i>			
Attore Principale	Amministratore			
Attori secondari	N.a.			
Entry Condition	Il sistema è attivo e l'amministratore può accedere alla macchina su cui è in esecuzione.			
Exit condition On success	Tutti i servizi vengono terminati nel modo previsto e la piattaforma risulta disattivata.			
Exit condition On failure	Almeno uno dei servizi non viene chiuso correttamente.			
FLUSSO DI EVENTI PRINCIPALE/MAIN SCENARIO				
1	Amministratore:	Richiede la chiusura del web container tramite comando dedicato.		
2	Sistema:	Esegue la procedura di terminazione del web container.		
3	Sistema:	Mostra una conferma dell'avvenuta chiusura del web container.		
4	Amministratore:	Richiede l'arresto del DBMS.		
5	Sistema:	Interrompe l'esecuzione del DBMS.		
6	Sistema:	Conferma che il DBMS è stato correttamente terminato.		
I Scenario/Flusso di eventi di ERRORE: Il web container non si arresta				
2.a1	Sistema	Esegue UC_BC_3 per gestire l'errore.		
Il Scenario/Flusso di eventi di ERRORE: Il DBMS non si arresta				
5.a1	Sistema	Attiva UC_BC_3.		

3. FALLIMENTO DEL SISTEMA

Identificativo <i>UC_BC_3</i>	<i>System Failure</i>		<i>Data</i>	<i>27/11/2025</i>
			<i>Vers.</i>	<i>0.00.001</i>
			<i>Autore</i>	<i>Team</i>
Descrizione	<i>Lo UC definisce cosa accade quando il sistema subisce un crash o un'interruzione imprevista.</i>			

Attore Principale	Sistema
Attori secondari	Amministratore
Entry Condition	Si verifica un arresto improvviso del sistema.
Exit condition On success	<i>Il sistema viene riavviato correttamente.</i>
Exit condition On failure	Il riavvio non va a buon fine.
FLUSSO DI EVENTI PRINCIPALE/MAIN SCENARIO	
1	Sistema: Rileva l'errore avvenuto e salva la sessione attuale.
2	Sistema: Invia una segnalazione all'utente per informarlo dell'anomalia.
3	Utente: Legge la comunicazione mostrata dal sistema.

4.ERRORE DI ACCESSO AI DATI PERSISTENTI

Identificativo <i>UC_BC_4</i>	<i>Errore di Accesso ai Dati Persistenti</i>	<i>Data</i>	<i>27/11/2025</i>
		<i>Vers.</i>	<i>0.00.001</i>
		<i>Autore</i>	<i>Team</i>
Descrizione	L'UC descrive la gestione degli errori sul database.		
Attore Principale	Amministratore		
Attori secondari	NA		
Entry Condition	Impossibilità di accedere ai dati o dati corrotti.		
Exit condition On success	Il Sistema torna al corretto funzionamento		
Exit condition On failure	Il sistema non può tornare operativo.		
FLUSSO DI EVENTI PRINCIPALE/MAIN SCENARIO			
1	Sistema:	Il Sistema informa l'Amministratore dell'impossibilità di leggere o utilizzare i dati persistenti.	
2	Sistema:	Il Sistema sospende il processamento di nuove richieste e risponde con messaggi di errore.	
3	Amministratore:	L'Amministratore avvia la procedura di spegnimento (UCBC_2).	
4	Amministratore	L'Amministratore ripristina la base dati o la rende accessibile	
5	Amministratore	L'Amministratore avvia nuovamente il sistema (UCBC_1).	

3.9 Servizi dei sottosistemi

La seguente tabella descrive i servizi offerti dai sottosistemi identificati nell'architettura proposta. Per ogni servizio viene fornita una breve descrizione e il componente software responsabile della sua erogazione.

Servizio	Descrizione	Interfaccia
Ricerca Veicoli	Il servizio permette di visualizzare l'elenco delle auto disponibili, applicando filtri per marca, prezzo e anno di immatricolazione.	CatalogoService
Dettaglio Veicolo	Il servizio restituisce la scheda tecnica completa di una specifica automobile, incluse foto e caratteristiche.	CatalogoService
Inserimento Veicolo	Il servizio consente all'amministratore di aggiungere una nuova automobile al catalogo, validando i dati inseriti.	CatalogoService
Rimozione Veicolo	Il servizio permette di eliminare un'automobile dal catalogo, rendendola non più visibile ai clienti.	CatalogoService
Richiesta Preventivo	Il servizio gestisce la creazione di una nuova richiesta di preventivo da parte del cliente per un veicolo specifico.	PreventivoService
Gestione Stato Richiesta	Il servizio permette al venditore di aggiornare lo stato di una richiesta (es. da "Nuova" a "Inviata" o "Respinta").	PreventivoService

Calcolo Rata Leasing	Il servizio fornisce una simulazione della rata mensile basata su anticipo, durata e chilometraggio inseriti dall'utente.	LeasingService
Richiesta Leasing	Il servizio formalizza la richiesta di leasing, salvando i dati della simulazione e notificando il venditore.	LeasingService
Autenticazione Utente	Il servizio verifica le credenziali (email e password) e garantisce l'accesso alla dashboard corretta in base al ruolo.	AutenticazioneService
Registrazione Cliente	Il servizio permette la creazione di un nuovo account cliente, verificando l'unicità dell'email e la correttezza dei dati.	RegistrazioneService
Gestione Profilo	Il servizio consente all'utente loggato di visualizzare e modificare i propri dati anagrafici.	ProfiloService